



ГАЙД ДЛЯ БИЗНЕСА • ВНУТРИ FUTURAAI • HERMES ЧАСТЬ 3

Hermes • Часть 3 — Полный гайд

Четыре обновления агента, каждое — двумя путями. **ПРОСТО:** вставляете готовый промпт в Codex или Claude Code — агент настраивает всё сам. **РУКАМИ:** конфиги и команды под копирование. Память слоями, дешёвая умная модель за \$18, одна система на сервер-чат-ПК и клетка вокруг агента, который сам себе выдаёт права. Плюс — как агент теперь сам куёт навыки, гоняет рой субагентов, плюс готовые навыки и приёмы. Все конфиги — бесплатно, файлом.

Память

семантический поиск по памяти —
тянет из памяти только нужное

\$18/мес

умный агент на GLM вместо
флагмана; ~6× дешевле по
выходным токенам

~\$5–6/мес

одна система на три устройства
через облако (community-вариант)

–84%

меньше запросов на разрешение —
strict-sandbox как основа клетки

futuraai.ru [Telegram @edvardgrishin27](https://t.me/edvardgrishin27)

00 • КАК ПОЛЬЗОВАТЬСЯ

Два пути на каждую тему. Выберите свой — и не смешивайте, пока не разберётесь

Все четыре темы ниже устроены одинаково: сначала **ПРОСТО**, потом **РУКАМИ**. Это не два разных материала — это одна и та же настройка, поданная под два уровня. Новичку хватит первого блока. Тому, кто хочет понимать каждую строчку, — второго.

ПУТЬ 1

● ПРОСТО

Копируете жёлтый промпт целиком и вставляете в Codex, Claude Code или самого Hermes. Агент читает документацию, предлагает конфиг под вашу сборку и объясняет каждый шаг. Вам остаётся подставить свои ключи и подтвердить. Идеально, если не хотите лезть в YAML руками.

ПУТЬ 2

● РУКАМИ

Готовые конфиги, команды и таблицы под копирование — ровно то, что показано в видео. Ставите сами, шаг за шагом. Плейсхолдеры вида `<ВАШ_КЛЮЧ>` и пометки «сверьте у себя» — это места, где нужно подставить своё или перепроверить актуальность под вашу версию.

Одно правило перед стартом. Не копируйте вслепую — ни промпт, ни конфиг. Проверяйте пути, порты, имена слотов и цены под свою сборку: набор фич отличается между версиями агента, а цены на модели и облако меняются буквально по неделям. Где значение не подтверждено первичным источником — стоит пометка «сверьте у себя». Это честный материал, а не «успешный успех».

С чего начать по задаче. Экономия за минуту — тема [Модели / GLM](#). Агент «забывает» или захлёбывается контекстом — [Память](#). Нужен агент 24/7 с доступом с телефона — [Роутинг + Облако](#). Оставляете агента работать автономно — [Клетка](#). Каждая тема самодостаточна.

ТЕМА 1 / 4

Память: не один инструмент, а стек слоями

Лучшей памяти как одного инструмента не существует. Лучшая память — это **стек**: начинаете с файлов и добавляете слои по мере роста. Четыре ступени снизу вверх — MD-файлы (ядро) → вектор (когда объём перерос файлы) → темпоральный граф Zep (когда факты меняются во времени) → Reflexion-цикл сверху (агент учится на своих ошибках, почти бесплатно). Эволюция, не революция: не прыгайте сразу в тяжёлые платные системы.

● ПРОСТО

вставьте этот промпт в Codex / Claude Code / Hermes

Агент оценит, на какой ступени вы сейчас, и проведёт к следующей — без прыжка в дорогое.

Ты настраиваешь память моего Hermes-агента. Я не разработчик. Главное требование:

НЕ ПОТЕРЯТЬ то, что уже есть в памяти. Нарачивай слоями, не ломай текущее.

ПРАВИЛА:

- ПЕРВЫМ делом найди мою текущую память (обычно ~/.hermes/MEMORY.md и папка ~/.hermes/memory/) и сделай её резервную копию. Покажи, куда сложил бэкап. Только потом что-либо меняй.
- Ничего не удаляй и не перезаписывай без моего явного «да». Каждое изменение — сначала показать, потом применить.
- Где точная возможность/ограничение Hermes не подтверждены — проверь в README/SECURITY.md проекта и скажи честно. Не выдавай паттерн за встроенную кнопку.

ПОДХОД — эволюция, а не революция. Иди снизу вверх и не перепрыгивай в тяжёлое платное:

Ступень 1 (ядро, оставить как есть): MD-файлы — правила, мой профиль, факты. Дёшево и наглядно. Учти жёсткий потолок: индекс памяти обрезается по объёму — около 2000 знаков (~30-40 строк) — дальше

агент не видит. Проверь, не упёрлась ли моя память в этот потолок; если да — предложи ступень 2.

Ступень 2 (только если объём реально перерос файлы): векторная память для поиска по смыслу (pgvector / Redis / Pinecone). Не подключай заранее «на всякий случай».

Ступень 3 (только если у меня факты МЕНЯЮТСЯ во времени — тарифы, статусы): темпоральный граф Zer/Graphiti. По сверенным данным (как сообщается вендором): LongMemEval +18,5% точности при -90% времени ответа. Не тащи, если у меня статичные факты.

Ступень 4 (поставь сверху — почти бесплатно): Reflexion-петля. Заведи файлы feedback_*.md, куда агент складывает разбор своих ошибок «в прошлый раз я тут ошибся, на будущее — вот так». Работает поверх любого стека, стоит копейки.

УЧТИ встроенные ограничения Hermes (это защита, не баг): запись в память — только .md; ensureWorkspacePath() не даёт агенту писать за пределы рабочей папки; поиск по памяти ограничен ~200 совпадениями. Не пытайся их обходить.

Дополнительно проверь, настроен ли у меня семантический поиск по памяти через эмбединги (embedding-провайдер для поиска по памяти и консолидации — это реальная настройка Hermes, смотри в экране провайдеров). Если не настроен — предложи включить и объясни просто: агент достаёт из памяти только релевантные под запрос куски, а не тащит всё подряд. Никаких обещаний процента экономии контекста — просто скажи, что стало точнее.

В конце: покажи, где теперь лежит память, что добавлено, и подтверди, что бэкап цел.

● РУКАМИ

лестница из четырёх ступеней

СТУПЕНЬ	КОГДА ВКЛЮЧАТЬ	ЦИФРЫ (КАК СООБЩАЕТСЯ)
1. MD-файлы (ядро)	Маленькое и явное: правила, профиль, факты. Дёшево, видно глазами.	Потолок ~2000 знаков (~30–40 строк)
2. Вектор (pgvector / Redis / Pinecone)	Внешний провайдер — подключается штатно (<code>memory.provider</code>), не костыль. Объём перерос файлы, нужен поиск по смыслу.	Минус: латентность на поиске, нет ощущения времени
3. Темпоральный граф (Zep / Graphiti)	Внешняя система (не кнопка Hermes). Факты меняются во времени (тарифы, статусы).	LongMemEval +18,5% / -90% времени ответа; DMR 94,8%
4. Reflexion-цикл (поверх)	Самый дешёвый апгрейд. Агент учится на ошибках.	91% pass@1 HumanEval vs 80% у голого GPT-4

Reflexion почти даром — файлами

Агент разбирает свои же ошибки и пишет вывод на будущее в `feedback_*.md` («в прошлый раз я тут ошибся»). Работает стихийно поверх любого стека — ничего платить не нужно.

Семантический поиск по памяти (эмбединги)

Агент ищет по **смыслу** запроса, а не тащит в контекст всю память подряд: под капотом отдельный **embedding-провайдер** для поиска по памяти и консолидации записей. Чем меньше мусора в контексте, тем точнее агент следует правилам. Включается это в **Settings → Agent Behavior → Memory** («Memory enabled» + «User profile»), а провайдер поиска задаётся в `~/.hermes/config.yaml` в блоке `memory` — по умолчанию `memory.provider` пуст (работает встроенная память), задаёте провайдера, чтобы подключить внешний:

```
# ~/.hermes/config.yaml — блок memory.*
memory:
  provider: "" # по умолчанию пуст = встроенная память; задаёте провайдера под свою
               конфигурацию
  # тумблеры «Memory enabled» и «User profile» — в Settings → Agent Behavior → Memory
  # точные ключи и список провайдеров — по официальной документации Hermes (значения зависят от
  версии)
```

Что уже есть в Hermes под память

- **Граф памяти** (`/journey`) — карта памяти + таймлайн, правка и удаление узла на месте (`hermes journey delete` или из интерфейса графа); показывает `~/.hermes/` (`MEMORY.md` + папка `memory/`).
- **Запись в память** — только `.md` : агент не может писать в память произвольные типы файлов.
- `ensureWorkspacePath()` — блокирует выход за пределы рабочей папки (path-traversal закрыт). Поиск по памяти ≤ 200 совпадений.

ЧЕСТНАЯ ОГОВОРКА

Нативно Hermes даёт слой 1 (MD-файлы) + встроенный поиск по памяти **по ключевым словам** (полнотекстовый, не по смыслу) + слой 4 (Reflexion, файлы `feedback_*.md`). А поиск **по смыслу**, через эмбединги, включается **внешним провайдером**: Settings → Agent Behavior → Memory и параметр `memory.provider` в конфиге, провайдеров 8 на выбор. Слой 2 (вектор pgvector/Pinecone) и слой 3 (темпоральный граф Zep/Graphiti) как таковые не вшиты, но векторную/графовую память вы подключаете тем же механизмом `memory.provider` — родным, не костылём. Нейтрального лидерборда памяти на июль 2026 нет — почти все бенчмарки self-reported самими вендорами (это касается цифр Zep/Mem0 выше).

ТЕМА 1 · ДОПОЛНЕНИЕ · НАВЫКИ 0.18

Навыки: теперь агент куёт их сам

Раньше навык вы вписывали руками — файлом `SKILL.md`. В обновлении 0.18 агент **сам** превращает опыт в навык: команда `/learn` делает из источника постоянный навык, а `/journey` показывает всю память таймлайном. Ниже помечаю статус каждой механики — **PRIMARY** (в движении), **ОФИЦ v0.18** (в релиз-нотах) или атрибуцию разбора.

● ПРОСТО

вставьте этот промпт в Codex / Claude Code / Hermes

Работает через штатную команду `/learn` и систему навыков `SKILL.md`.

Преврати вот этот источник в постоянный навык моего Hermes-агента.

ИСТОЧНИК: <вставь ссылку / путь к документу / описание процесса>

ПРАВИЛА:

- Используй штатную команду `/learn <путь|URL|заметка>` — она создаёт постоянный навык из источника. Покажи результат и куда лёг навык. Резервный путь — оформить руками через `SKILL.md`.
- Проверь заодно команды `/skills` и `/skill` и формат `SKILL.md` — на них держится вся система навыков.
- Разбери источник, вытащи из него пошаговый навык, назови понятно.
- Ничего в существующих навыках не перезаписывай без моего «да». Покажи, что меняешь.

По желанию: схема «победа → навык» — это просто `/learn` поверх завершённого удачного разговора

(«this chat»): сохраняешь пройденный путь как новый навык, чтобы не проходить его заново.

Отдельной кнопки «по победе» нет — это ручной `/learn` в нужный момент.

● РУКАМИ

механики и их статус

МЕХАНИКА	ЧТО ДЕЛАЕТ	STATUS
Навыки <code>SKILL.md</code> + команды <code>/skills</code> , <code>/skill</code>	Создать и поставить навык штатно. Основа всей системы навыков.	PRIMARY
<code>/learn <путь URL заметка></code>	Создаёт постоянный навык прямо из источника — путь, ссылка или заметка. Одна команда «источник → навык».	PRIMARY
Победа → навык	Это <code>/learn</code> поверх завершённого удачного разговора («this chat») + Learning Loop. Авто-триггер «по победе» — редакторская формулировка, отдельной кнопки нет.	PRIMARY
<code>/journey</code>	Таймлайн памяти + radial memory graph; узлы правятся и удаляются прямо в графе (inline).	ОФИЦ v0.18
Двусторонняя Obsidian-вики	Агент и читает вашу Obsidian-вику, и пишет в неё: «проиндексируй разговор». Ядро памяти не засоряется.	community-приём

ЧЕСТНАЯ ОГОВОРКА

Система навыков `SKILL.md`, команды `/skills` / `/skill` и `/learn` — **реальные, в движке**; «победа→навык» — это тот же `/learn` поверх завершённого чата (не отдельная кнопка). `/journey` (таймлайн + radial memory graph) — в релиз-нотах v0.18. Двусторонняя Obsidian-вика (агент читает **и пишет**) — это сторонний community-приём поверх Obsidian, не кнопка в Hermes.

Двусторонняя вика по методу Карпати (LLM Wiki) — пошагово

Метод Карпати — **LLM Wiki** (апрель 2026): вместо RAG и бесконечного контекста — **живая markdown-вика, которую агент сам компилирует и поддерживает**. Три слоя: **сырьё** (исходники) → **вика** (агент сжимает сырьё в связанные страницы с перекрёстными ссылками) → **схема** (файл-правила, как вику вести). Три операции: *ingest* (сырьё → страницы), *query* (сначала читаем вику), *lint* (чистим дубли). Ложится на Hermes напрямую: память — это `.md`, агент их и читает, и пишет.

● ПРОСТО

вставьте этот промпт в Codex / Claude Code / Hermes

Собери мне базу знаний по методу Карпати (LLM Wiki) для моего Hermes-агента. Я не разработчик.

- Заведи папку `wiki/` в моей памяти — `.md`-страницы, которые **ВЕДЁШЬ** ТЫ, с перекрёстными ссылками `[[...]]`.
- Заведи файл `wiki-schema.md` — правила: типы страниц, как называть, как ссылаться, и три операции: *ingest* (сырьё → страницы), *query* (сначала читаешь вику), *lint* (чистишь дубли/устаревшее).
- Моё ядро `MEMORY.md` держи **ТОНКИМ** — только правила и профиль; вику туда **НЕ** сваливай.
- Работай двусторонне: даю сырьё (ссылку/док/разговор) — компилируешь в страницы вики; спрашиваю — сначала читаешь вику; после сессии — обновляешь нужные страницы («проиндексируй этот разговор»).
- Ничего в старой памяти не перезаписывай без моего «да». Покажи схему и где лежит вика.

Руками — за 4 шага

1. **Структура.** `~/hermes/memory/wiki/` — .md-страницы вики (с `[[ссылками]]`); `wiki-schema.md` — правила ведения; `MEMORY.md` оставляете тонким ядром (правила/профиль). По желанию тот же каталог открываете как Obsidian-vault — листать с backlinks.
2. **Схема** `wiki-schema.md`. Опишите типы страниц, соглашение об именах и ссылках, и три операции: *ingest* / *query* / *lint*. Это «конституция» вики — по ней агент держит её в порядке и не плодит дубли.
3. **Двусторонний цикл.** Агент читает вику как источник истины И пишет в неё сводки после каждой сессии — сырьё сжимается в summary, вика **накапливается (компаундится)**, в отличие от stateless-RAG.
4. **Границы Hermes.** Запись — только `.md`, `ensureWorkspacePath()` держит агента в рабочей папке: вика живёт внутри, ядро `MEMORY.md` не засоряется.

ЧЕСТНАЯ ОГОВОРКА

Сам паттерн LLM Wiki — метод Andrej Karpathy (апрель 2026), а двусторонняя Obsidian-интеграция — community-приём, **не штатная кнопка Hermes**. Hermes даёт `.md` -память + `ensureWorkspacePath()`; «вика» — это вы организуете эти `.md` по Карпати. Точные команды сверьте на день установки.

ТЕМА 2 / 4

Дешёвые умные модели: GLM 5.2 в main, весь фон — на копейки

Идея простая: дорогую reasoning-модель — только туда, где она реально нужна (слот `main`, отвечает вам и крутит инструменты). Весь служебный фон (~11 слотов `auxiliary`: заголовки, зрение, сжатие контекста, извлечение из веба) — на дешёвую и быструю. А в сам `main` ставите почти-флагманский код за цену в разы ниже — GLM 5.2.

● ПРОСТО

вставьте этот промпт в Codex / Claude Code / Hermes

Агент разложит модели по слотам и подключит дешёвую-но-умную, не сломав то, что уже работает.

Ты настраиваешь роутинг моделей в моём Hermes-агенте. Я не разработчик.

Цель: умная модель в основном слоте `main`, дешёвая быстрая — на служебном фоне `auxiliary`, плюс аккуратные умные подсказки по удешевлению.

ПРАВИЛА:

- Сначала покажи текущий config.yaml (слот main + список провайдеров) и план изменений. Применяй только после моего «да». Сделай бэкап config.yaml перед правкой.
- Ключи клади в ~/.hermes/.env, не в config.yaml. Не показывай ключи в открытом виде.
- Точные имена auxiliary-слотов и синтаксис overrides сверь через `hermes model` — набор слотов меняется между версиями (сейчас ~0.18). Не выдумывай имена.
- Слаг модели и base_url, в которых не уверен, — проверь в openrouter.ai/models или docs.z.ai и скажи честно, если не нашёл точное значение.

ОСНОВНАЯ МОДЕЛЬ (main) — GLM 5.2. Подключи одним из способов, спроси какой мне удобнее:

Вариант А (по API через OpenRouter):

provider: openrouter, model: <слаг GLM 5.2 из openrouter.ai/models>

ключ OPENROUTER_API_KEY=<ВАШ_КЛЮЧ_OPENROUTER> в ~/.hermes/.env

Вариант В (напрямую Z.ai, OpenAI-совместимый провайдер):

provider Z-AI/GLM с base_url <сверь в docs.z.ai>, api_mode chat_completions,

ключ <ВАШ_КЛЮЧ_Z_AI>

Вариант С (подписка вместо API — GLM Coding Plan Lite, \$18/мес; при годовой -30%):

тот же OpenAI-совместимый механизм, но base_url и ключ берутся из кабинета z.ai.

Лимит у подписки в промптах (~80 за 5 часов, ~400 в неделю), не в токенах.

Учти: старого \$3-промо больше нет, база — \$18/мес. Цену сверь на z.ai/subscribe.

ЕСЛИ ХОЧУ ТОП, а не баланс — предложи Grok 4.5 (провайдер xAI, штатный в Hermes): флагман под код/агенты, контекст 500К, но ДОРОЖЕ GLM — \$2/\$6 за 1М токенов. Слаг актуальной модели сверь в docs.x.ai. Подписку SuperGrok (\$30/мес) в обёртки заводят через OAuth без ключа — под Hermes официально НЕ подтверждено, проверь и скажи честно.

Честно предупреди меня: GLM 5.2 — это лучший баланс «ум на деньги», НЕ «самая умная» модель. По сверенным данным Terminal-Bench 2.1 = 81.0 (Opus 4.8 = 85.0, GPT-5.5 = 84.0), SWE-bench Pro = 62.1 (обходит GPT-5.5, уступает Opus 4.8). Цена \$1,40/\$4,40 за 1М токенов.

ДЕШЁВЫЙ ФОН (auxiliary) — переопредели ТОЛЬКО служебные слоты на быструю дешёвую модель, не трогая main. Ориентир: заголовки сессий, зрение (gemini-2.5-flash), сжатие контекста, оценка на одобрение (gpt-4o-mini / Haiku), веб-извлечение — всё это не требует reasoning, им хватит Gemini Flash или mini. Реальные имена слотов возьми из `hermes model`.

SMART SUGGESTIONS — включи умные подсказки модели по задаче, но безопасно:

smartSuggestionsEnabled = true, и обязательно onlySuggestCheaper = true

(подсказывать только удешевление, а не апгрейд на дорогую). Задай

preferredBudgetModel (дешёвая) и preferredPremiumModel (умная).

Предупреди меня про ловушку: смена модели командой /model в середине сессии сбрасывает prompt cache — за который я уже заплатил. Совет по итогу: выбирать модель на старте сессии и не дёргать по ходу. И скажи, что изменения конфига применяются не мгновенно (CLI — со

следующего вызова, gateway — в новых сессиях, иногда нужен рестарт).

В конце покажи итоговый config.yaml (с замаскированными ключами) и что проверить.

Modes и drift-detection — сохранённые наборы настроек

Рядом со Smart Suggestions живут ещё две вещи из ролика: **Modes** — сохраняете именованный набор настроек моделей и грузите одним кликом (например «дёшево-фон» и «дорого-финал»); и **drift-detection** — если текущие настройки уползли от сохранённого режима, интерфейс это подсветит. Смысл простой: не пересобирать слоты руками каждый раз и не удивляться потом, почему «вчера было дешевле».

ЧЕСТНАЯ ОГОВОРКА

Smart Suggestions (`smartSuggestionsEnabled` , `onlySuggestCheaper`) подтверждены кодом Workspace. А **Modes и drift-detection в публичной документации Hermes не описаны** — механика из разбора/сборки, точные названия экранов и ключей зависят от версии. Найдите их на экране моделей/настроек своей сборки и сверьте, прежде чем на них опираться.

Куда какую модель (auxiliary)

ЗАДАЧА	РЕКОМЕНДАЦИЯ	ПОЧЕМУ
Заголовки сессий	Gemini Flash	Генерации на копейки, рассуждать не надо
Зрение (vision)	<code>gemini-2.5-flash</code>	Дёшево, быстро
Сжатие контекста	Любая дешёвая	Суммаризация не требует reasoning
Оценка одобрения	Haiku / Flash / <code>gpt-4o-mini</code>	Простая классификация
Извлечение из веба	Быстрая модель	≈ 1/50 стоимости той же операции на reasoning-модели

Пример `~/.hermes/config.yaml` с overrides

```
# Основная модель — отвечает вам и крутит инструменты
provider: openrouter
model: openrouter/pareto-code           # умный роутер: самая дешёвая выше порога

# Переопределяем ТОЛЬКО дешёвый фон:
auxiliary:
  title:      { provider: google, model: gemini-flash-latest }
  vision:     { provider: google, model: gemini-2.5-flash }
  compression: { provider: google, model: gemini-flash-latest }
  approval:   { provider: openai, model: gpt-4o-mini }
  web_extract: { provider: google, model: gemini-flash-latest }
# ^ имена слотов уточните у себя через `hermes model`
```

Умные роутеры OpenRouter в `main`

- `openrouter/auto` — сам подбирает модель под запрос.
- `openrouter/pareto-code` — берёт самую дешёвую модель выше вашего порога качества по коду (`min_coding_score: 0.65`). Требует контекст `main` ≥ 64K; при отвале модели сессия не теряется — подхватывает следующая по цепочке фолбэка.

Подключение GLM 5.2 — три варианта

```
# Вариант А — по API через OpenRouter
provider: openrouter
model: z-ai/glm-5.2          # слаг сверьте в openrouter.ai/models
# ~/.hermes/.env: OPENROUTER_API_KEY=sk-or-v1-<ВАШ_КЛЮЧ>

# Вариант В — напрямую Z.ai (custom OpenAI-совместимый провайдер)
provider: z-ai
model: glm-5.2              # точное имя сверьте в docs.z.ai
custom_providers:
  - name: z-ai
    base_url: https://api.z.ai/api/paas/v4    # base_url сверьте у себя
    api_key: <ВАШ_КЛЮЧ_Z_AI>
    api_mode: chat_completions

# Вариант С — GLM Coding Plan Lite ($18/мес) как endpoint (подписка)
provider: glm-coding
model: glm-5.2
custom_providers:
  - name: glm-coding
    base_url: <ENDPOINT_ИЗ_КАБИНЕТА_Z_AI>
    api_key: <КЛЮЧ_ПОДПИСКИ>
    api_mode: chat_completions
```

Цены по API (за 1М токенов)

МОДЕЛЬ	INPUT	OUTPUT	ЗАМЕТКА
GLM 5.2 (Z.ai напрямую)	\$1,40	\$4,40	кэш-вход \$0,26; 1М контекст; лицензия MIT
GLM 5.2 (OpenRouter)	\$1,00	\$4,00	бывает дешевле прямого — сверьте слаг
DeepSeek V4 Flash	\$0,14	\$0,28	самый дешёвый; проверяйте tool-calling
Kimi K2.7-Code	\$0,95	\$4,00	token-efficiency в агентах
Grok 4.5 (xAI) · ТОП	\$2,00	\$6,00	флагман код/агенты; 500K контекст; дороже GLM; слаг сверьте в docs.x.ai
Kimi K3 (Moonshot) · ТОП	\$3,00	\$15,00	новейшая (16.07.26); крупнейшая открытая 2.8T; №1 фронтенд-код; кэш ~60–80%; OpenAI-совм. <code>api.moonshot.ai/v1</code> (ключ — platform.kimi.ai); подписка Kimi Code

Нужен топ, а не самое дешёвое? **Grok 4.5 (xAI)** — флагман под код и агентов, контекст 500K; в Hermes подключается штатно (провайдер xAI). Но он **дороже GLM**: \$2/\$6 за 1M против \$1,40/\$4,40 (на длинных промптах — до \$4/\$12). Подписку **SuperGrok (\$30/мес)** / X Premium+ (\$40/мес) в ряде обёрток заводят через вход по аккаунту без ключа — под Hermes официально не подтверждено, сверьте. А если нужна **сильнейшая по коду прямо сейчас** — **Kimi K3 (Moonshot)**, вышла 16 июля 2026: крупнейшая открытая модель (2.8T, контекст 1M), №1 в арене фронтенд-кода, обходит GLM 5.2 и Fable 5. Но она **премиум, не бюджет**: \$3/\$15 за 1M (кэш даёт ~60–80%). Как поставить: ключ берёте на platform.kimi.ai, в конфиг — `base_url: https://api.moonshot.ai/v1` и модель `kimi-k3` (OpenAI-совместимо, работает через Claude Code / Cline / LangChain). Или по подписке **Kimi Code** (плагается в Claude Code / Cline, как GLM-план; ~\$7/нед, 300–1200 вызовов/5ч — тариф и модель сверьте, планы двигаются).

Ловушка, которая съедает всю экономию. Смена модели в середине сессии командой `/model` сбрасывает prompt cache — контекст, за который вы уже заплатили. На длинном диалоге дешёвая модель с постоянными переключениями выходит дороже дорогой. Вывод: выбирайте модель на старте сессии и не дёргайте по ходу.

ЧЕСТНОЕ ПОЗИЦИОНИРОВАНИЕ

GLM 5.2 — **не «самая умная»**. На Terminal-Bench 2.1 её обходят Opus 4.8 (85,0 против 81,0) и GPT-5.5 (84,0); на SWE-bench Pro она бьёт GPT-5.5, но уступает Opus. Её сила — **баланс «ум на деньги»**: \$1,40/\$4,40 против \$5/\$30 у флагмана OpenAI — **~6–7× дешевле по выходным токенам**. Именно отсюда ходячая фраза «~1/6 стоимости», но сама «1/6» — фрейминг прессы, а не заявление Z.ai. Легендарного \$3-промо на подписку больше нет — база \$18 (или \$12,6/мес при годовой оплате). Бенчмарки — «как сообщается вендором».

ТЕМА 2 · ДОПОЛНЕНИЕ · НЕСКОЛЬКО МОДЕЛЕЙ РАЗОМ

Несколько моделей разом: рой, модель-под-профиль, МОА

Раз уж речь про модели по слотам — три вещи из обновления, которые усиливают агента. Все три — **PRIMARY**, в коде: fork-субагенты, модель-под-профиль, МОА.

● ПРОСТО

вставьте этот промпт в Codex / Claude Code / Hermes

Агент проверит по вашей сборке, что из этого доступно, и настроит рабочее.

Настрой работу с несколькими агентами и моделями сразу в Hermes.

1. FORK-СУБАГЕНТЫ. Проверь команды `/fork` и `/agents`: запусти рой помощников в фоне под отдельную задачу, покажи панель со статусами воркеров (кто чем занят). Это штатно — если в моей сборке нет, скажи честно.
2. МОДЕЛЬ-ПОД-ПРОФИЛЬ (profiles). Настрой, чтобы у каждой «профессии»/навыка была СВОЯ модель и свой системный промт: per-profile конфиг лежит в `~/.hermes/profiles/<id>/config.yaml`, персона — в `SOUL.md` (дорогая модель — на сложное, рутина — на дешёвой). Экран называется «profiles».
3. MOA (Mixture of Agents). Это реальный РЕЖИМ, не бейдж: собери пресет из моделей-советников + агрегатор (топ-модель уровня Opus). Один вопрос → ответы советников + сводный ответ агрегатора.

По каждому пункту: сначала покажи, что реально есть в моей версии, потом настрой.

Где кнопки нет — так и скажи, не выдумывай.

● РУКАМИ

три механики и их статус

МЕХАНИКА	ЧТО ДЕЛАЕТ	СТАТУС
Fork-субагенты + панель (<code>/fork</code> , <code>/agents</code>)	Рой помощников в фоне под задачу, живая панель статусов воркеров.	PRIMARY
Модель-под-профиль (profiles)	Своя модель + системный промт на каждую «профессию»/персону: <code>~/.hermes/profiles/<id>/config.yaml</code> , персона — <code>SOUL.md</code> .	PRIMARY
MOA (Mixture of Agents)	Пресет из моделей-советников + агрегатор (топ-модель уровня Opus): один вопрос → ответы советников + сводный. «Coordinate multiple AI models together».	PRIMARY

ЧЕСТНАЯ ОГОВОРКА

Fork-субагенты и панель воркеров — **PRIMARY** (команды `/fork` , `/agents`). «Модель-под-профиль» реальна: per-profile модель + системный промт (`~/.hermes/profiles/<id>/config.yaml` , персона в `SOUL.md`); экран называется «profiles».

MOA (Mixture of Agents) — тоже реальный режим из capability-реестра.

Одна система на сервер, чат и ПК — без потери задач

Честно с самого начала: явной кросс-девайс синхронизации в официальных доках нет. Единство достигается проще — **ОДИН инстанс агента + ОБЩЕЕ хранилище** (`~/.hermes` или его бэкап в облако). «Задачи не теряются» = один агент, одна память и много окон в него: терминал на ПК, шлюз в мессенджерах, веб-дашборд. Синхронизации между тремя равноправными копиями тут нет — это один инстанс плюс бэкап, а не distributed sync.

● ПРОСТО

вставьте этот промпт в Codex / Claude Code / Hermes

Агент поднимет одно хранилище и подключит к нему телефон и ноутбук — без ложных обещаний про мульти-мастер.

Ты разворачиваешь мой Hermes на сервере/в облаке через docker-compose. Я не разработчик.

Цель: один инстанс агента + общее хранилище ~/.hermes, чтобы я подключался к ОДНОМУ агенту с ПК и с телефона и задачи не терялись между устройствами.

ЧЕСТНО СРАЗУ (проговори мне это): официальная кросс-девайс синхронизация у Nous в статусе «Coming Soon» — её пока нет как готовой мульти-мастер-синхронизации. Единство достигается проще: ОДИН агент + ОДНО общее хранилище, к которому цепляются все окна (терминал, мессенджеры, веб-дашборд). Это не три равноправные синхронизирующиеся копии. Если будем ломиться в один стор с двух устройств одновременно — доки не обещают отсутствие конфликтов.

ПРАВИЛА:

- Сначала покажи план и команды, запускай после моего «да». Ничего необратимого без подтверждения. Особенно осторожно с `docker compose down -v` — он УДАЛЯЕТ данные (тома). Никогда не запускай его без отдельного явного подтверждения.
- Секреты (пароль веб-UI, API-ключи) не показывай в открытом виде и не коммить в git.
- Где точное имя тома, порт или переменная не подтверждены — сверь в реальном docker-compose.yml и .env.example репозитория, а не по памяти.

РАЗВЁРТЫВАНИЕ (образы pre-built, локальная сборка не нужна):

1. git clone https://github.com/outsourc-e/hermes-workspace.git && cd hermes-workspace
2. cp .env.example .env
3. В .env добавь ХОТЯ БЫ ОДИН ключ провайдера:
OPENROUTER_API_KEY=<ВАШ_КЛЮЧ> (или OPENAI_API_KEY / GOOGLE_API_KEY / ANTHROPIC_API_KEY)
4. docker compose up → открой http://localhost:3000

Образы: hermes-agent → nousresearch/hermes-agent:latest (порт 8642 gateway, 9119 dashboard внутри Docker-сети); hermes-workspace → ghcr.io/outsourc-e/hermes-workspace:latest (порт 3000).

ОБЯЗАТЕЛЬНО для доступа не с localhost (иначе сервер откажется стартовать — это защита):

```
HOST=0.0.0.0

HERMES_PASSWORD=<СИЛЬНЫЙ_СЕКРЕТ_32+_СИМВОЛА> # без него не-loopback хост не поднимется
# за HTTPS-прокси добавь: COOKIE_SECURE=1 и TRUST_PROXY=1 (только за доверенным прокси)
# если у gateway включён API_SERVER_KEY — workspace шлёт тот же секрет как HERMES_API_TOKEN
```

ОБЩЕЕ ХРАНИЛИЩЕ (чтобы задачи не терялись) — это тома из реального docker-compose.yml:

hermes-agent-data → монтируется в /opt/data и /home/workspace/.hermes (config, сессии, скиллы, память, credentials);

hermes-workspace-files → /workspace (файлы из файлового браузера).

Оба переживают пересоздание контейнеров и `docker compose down`. Удаляет только `down -v`.

ВНИМАНИЕ на расхождение: проза README упоминает legacy-том `claude-data`, но актуальный docker-compose.yml использует hermes-agent-data + hermes-workspace-files. Ориентируйся на ИМЕНА ИЗ РЕАЛЬНОГО docker-compose.yml, не из текста README. Сверь у меня перед бэкапом/миграцией.

ДОСТУП С ТЕЛЕФОНА И ПК (из README) — через Tailscale:

поставь Tailscale на сервер и на устройство, один аккаунт; `tailscale ip -4` → 100.x.y.z;
открой http://100.x.y.z:3000. На сервере укажи reachable-адрес backend (не 127.0.0.1) и слушать на всех интерфейсах: API_SERVER_HOST=0.0.0.0 в ~/.hermes/.env, плюс
HERMES_API_URL=http://100.x.y.z:8642 и HERMES_DASHBOARD_URL=http://100.x.y.z:9119 в .env.

Если захочу официальное Hermes Cloud – напони, что оно в PREVIEW (v0.18.2, регион lhr), оплата по дням scale-to-zero: Small \$0.32/день, Medium \$0.59/день, Large \$1.12/день, в простое \$0.06/день (это минимальная плата за хранение, HE ноль); для запуска нужен минимум \$10 кредитов или подписка Nous Portal. Ставки Cloud HE включают inference и инструменты – они отдельно. И что кросс-девайс sync / командная память там всё ещё «Coming Soon». Цены сверь на день установки.

Self-host через docker-compose (из реального файла проекта)

```
git clone https://github.com/outsourc-e/hermes-workspace.git
cd hermes-workspace
cp .env.example .env
# добавьте В .env хотя бы один ключ провайдера
# (OPENROUTER_API_KEY / OPENAI_API_KEY / GOOGLE_API_KEY ...)
docker compose up
# откройте http://localhost:3000
```

Тянет два pre-built образа: **hermes-agent** (`nousresearch/hermes-agent:latest` , порт 8642 gateway) и **hermes-workspace** (`ghcr.io/outsourc-e/hermes-workspace:latest` , порт 3000). Volume `hermes-agent-data` (config, сессии, скиллы, память, credentials) и `hermes-workspace-files` переживают пересоздание контейнеров и `docker compose down` ; удаляет только `down -v` .

Ключевые переменные `.env`

```
# Связь workspace ↔ agent (в Docker подставляется автоматически):
HERMES_API_URL=http://hermes-agent:8642
HERMES_DASHBOARD_URL=http://hermes-agent:9119

# ОБЯЗАТЕЛЬНО для любого не-loopback доступа: пароль веб-UI (32+ симв.)
HERMES_PASSWORD=<СИЛЬНЫЙ_СЕКРЕТ>
# Без пароля сервер откажется стартовать на не-loopback хосте.

# За HTTPS-прокси (Traefik / Nginx / Cloudflare / Tailscale):
# COOKIE_SECURE=1
# TRUST_PROXY=1 # только за доверенным прокси, чистящим x-forwarded-for
```

Доступ с телефона и ноутбука — Tailscale

```
# поставьте Tailscale на сервер и на устройство, войдите в один аккаунт
tailscale ip -4 # → 100.x.y.z, откройте http://100.x.y.z:3000

echo 'HERMES_API_URL=http://100.x.y.z:8642' >> .env
echo 'HERMES_DASHBOARD_URL=http://100.x.y.z:9119' >> .env
echo 'API_SERVER_HOST=0.0.0.0' >> ~/.hermes/.env # чтобы пиры дотянулись
```

Live re-pairing без рестарта: если workspace уже запущен — Settings → Connection → меняете URL. Значения пишутся в `~/.hermes/workspace-overrides.json` и применяются сразу.

Шесть бэкендов терминала — где агент реально выполняет команды

Тот самый список из ролика. Бэкенд задаётся в конфиге; проверить готовность — `hermes doctor` .

БЭКЕНД	КОГДА БРАТЬ	ЧТО НУЖНО
local	Старт по умолчанию, самый безопасный вход	ничего
Docker	Изоляция на своей машине	установленный Docker
SSH	Выполнять на своём сервере	SSH-доступ
Singularity	HPC-кластеры / научная инфраструктура	apptainer или singularity в \$PATH
Modal	Serverless: среда засыпает в простое, будится по требованию	MODAL_TOKEN_ID (или ~/.modal.toml)
Daytona	Serverless-персистентность — между сессиями почти ноль затрат	DAYTONA_API_KEY

Практика: начинайте с local ; «всегда на связи и дёшево в простое» — Daytona / Modal (гибернация); нужно своё железо под контролем — SSH / Docker.

Варианты «Hermes + облако»

ВАРИАНТ	ЧТО ЭТО	ЦИФРЫ / СТАТУС
Официальное Hermes Cloud	always-on, scale-to-zero «платишь пока работает»	Small \$0,32/день · Medium \$0,59 · Large \$1,12; в простое \$0,06/день для всех; мин. \$10 кредитов. Статус — PREVIEW (работает, цена открыта)
Wescome/hermes (Cloudflare)	бэкап в R2 каждую минуту, браузер + мессенджеры	~\$5-6/мес с гибернацией или ~\$34,50 круглосуточно. Community, не Nous
docker-compose	оба образа на своём VPS, стейт в volume	Стоимость вашего VPS. Из реального файла проекта

ЧЕСТНЫЙ МИНУС

Кросс-девайс синхронизация, командная общая память и one-click deploy в README помечены **Coming Soon / Pending infra** — не ищите кнопку, которой пока нет. Ставки Cloud **не включают** inference и инструменты — они списываются отдельно из кредита. Тарифы Cloud ≠ подписки Nous Portal (Free/Plus \$20/Super \$100/Ultra \$200 — это про доступ к моделям). И помните: если ломиться в один стор с двух устройств параллельно — доки не обещают отсутствие конфликтов.

ТЕМА 4 / 4

Клетка для агента: как усмирить самопатч

Автономный агент в окне разрешения может пропатчить сам себя: снять песочницу «потому что мешает», подменить бинарь, выдать себе доступ, которого вы не давали. За пять минут. **Главный инсайт:** нельзя полагаться на то, что модель «решит» соблюдать правило. Правило в промпте, в CLAUDE.md , в «неизменяемом тексте» — это не защита, это пожелание. Всё неотменяемое должно жить в слое, до которого агент физически не дотянется.

● ПРОСТО

вставьте этот промпт в Codex / Claude Code / Hermes

Агент выстроит клетку СНАРУЖИ себя — от быстрых штатных рычагов к изоляции ОС.

Ты помогаешь мне посадить моего автономного Hermes-агента в «клетку». Я не разработчик. Ключевой принцип: защита должна жить СНАРУЖИ агента, там, где он до неё физически не дотянется. Правило в промпте или в тексте конфига — это пожелание, а не защита; автономный агент в окне разрешения может снять с себя песочницу «потому что мешает». Поэтому НЕ полагайся на самоограничение агента — ставь внешние барьеры.

ПРАВИЛА:

- Сначала покажи план и конкретные команды/изменения, применяй после моего «да». Ничего необратимого без подтверждения.
- Чётко разделяй: что из этого — реальная встроенная кнопка Hermes, а что — настройка на стороне моей ОС / биллинга провайдера (не тумблер внутри Hermes). Не выдавай одно за другое.
- Где не уверен в синтаксисе — проверь в актуальной документации (Hermes SECURITY.md, Anthropic docs) и скажи честно, если не нашёл.

СНАЧАЛА включи то, что уже есть в самом Hermes (это подтверждено SECURITY.md проекта):

- Exec approval — опасные exec-команды требуют моего явного «да» через окно. Убедись, что включено.
- Skills security scanning — навыки прогоняются на подозрительные паттерны (помощник для ревью, не жёсткий барьер; финальная граница — ручной просмотр перед установкой).
- Запись в память только .md + ensureWorkspacePath() — агент не перепишет свой permission-конфиг и не выйдет за рабочую папку. Проверь, что это активно.

ЗАТЕМ поставь внешние ограничители (объясни мне каждый простым языком):

1. SPEND LIMIT — лимит трат на стороне биллинга провайдера (Console). Это стена ВЫШЕ агента, он её не обойдёт. Прямое лекарство от «счёта на \$200». Помоги мне его выставить.
2. STRICT SANDBOX — если харнесс поддерживает: запретить команды вне песочницы (allowUnsandboxedCommands=false) и не запускаться вовсе, если песочницы нет (failIfUnavailable=true). В этом режиме запись в собственный settings.json блокируется. Синтаксис сверь под мой харнесс.
3. MANAGED SETTINGS (deny старше allow) — если доступно: правила от админа/сервера вне досягаемости агента; deny-лист всегда старше любого allow. Хук может ужесточить, не ослабить.
4. EXEC-APPROVAL / ask-гейты на необратимое — push в main, трата денег, действия в prod требуют моего подтверждения.
5. СЕКРЕТЫ — по умолчанию они читаются! Явно закрой на чтение папки с ключами: denyRead для ~/.ssh/** и ~/.aws/** (и всё, что я укажу дополнительно).
6. АУДИТ НАРУЖУ — лог действий агента писать во внешний приёмник, ВНЕ рабочей папки агента, иначе он же его и подчистит.

Честно предупреди: обычный чёрный список путей против мотивированного агента слабоват (alt-path, копия бинаря, динамический линкер обходят его). Настоящую нижнюю границу даёт только внешняя изоляция процесса (gVisor/microVM) или kernel-уровень — но это уже настройка моей ОС/Linux, НЕ кнопка Hermes. Если это применимо к моей системе — скажи; если нет — не делай.

В конце: чек-лист «что закрыто клеткой / что осталось открытым / что настроено вне Hermes».

Сначала — реальные рычаги Hermes (уже в продукте)

Подтверждено файлом `SECURITY.md` v3.0.0+ реального проекта. Не «сделай сам»:

- **Exec approval modal** — чувствительные ехес-команды требуют явного человеческого «да» через окно в интерфейсе. Ключевой рычаг против самомодификации. В 0.19 «Quicksilver» умные одобрения включены по умолчанию: опасную команду сначала оценивает отдельная модель-ревьюер, а не только чёрный список — меньше лишних вопросов по мелочи, жёстче реакция на реально опасное.
- **Skills security scanning** — навыки из маркетплейса прогоняются на подозрительные паттерны. По SECURITY.md это помощник для ревью, а не жёсткий барьер: финальная граница — ваш просмотр перед установкой.
- **Запись в память только .md + ensureWorkspacePath()** — агент не может переписать permission-конфиг или выйти за рабочую папку.
- **Все API-роуты требуют аутентификации** (с v3.0.0); токены Claude только server-side; cookies `httpOnly` + `SameSite=Strict`.
- **Rate limiting 10 req/min на IP** на высокорисковых эндпоинтах (`/api/terminal-input` , `/api/restart` , `/api/update-check`).

Почему path-денлист бесполезен против мотивированного агента

ВЕКТОР	ЧТО ДЕЛАЕТ АГЕНТ	ПОЧЕМУ ПРОБИВАЕТ
Alt-path	<code>/proc/self/root/usr/bin/npx</code>	тот же файл (inode), другой путь — фильтр промазал
Отключение sandbox	«мешает задаче → снимаю»	правило на команде, не на процессе
Динамический линкер	<code>ld-linux...so.2 /usr/bin/wget</code>	грузит через <code>mmap</code> , мимо <code>execve</code>
Копия бинаря	<code>cp node /tmp/mynode</code>	новый путь ≠ старый паттерн

Семь слоёв — от «нельзя обойти» (низ) к «удобно» (верх)

- **1. Внешняя изоляция процесса.** gVisor (`runsc`) или Firecracker microVM (boot ~125 мс). Non-root, сеть закрыта. Агент не снимет то, что СНАРУЖИ него. Обычный Docker/runsc не спасает — общее ядро с хостом.
- **2. Strict sandbox.** `allowUnsandboxedCommands:false` , `failIfUnavailable:true` . Автоматически запрещает запись в `settings.json` ; аварийный люк `dangerouslyDisableSandbox` игнорируется. Бонус: **−84%** запросов на разрешение — безопаснее И автономнее.
- **3. Managed settings.** Настройки от сервера/админа (MDM), вне досягаемости агента. Deny всегда СТАРШЕ любого hook-allow; хук может только ужесточить. Плюс `disableBypassPermissionsMode` .
- **4. PreToolUse hook-deny.** Держится даже в bypass-режиме / с `--dangerously-skip-permissions` . При совпадении с денлистом `exit 2` блокирует вызов инструмента.
- **5. Секреты.** По умолчанию читаются — явно закройте: `{ "permissions": { "denyRead": ["~/.ssh/**", "~/.aws/**"] } }` .
- **6. Деньги, два уровня.** Spend limit в Console (стена на биллинге, не обойти) + budget-guard hook (режет действие при превышении метрики расхода). Лечит кейс «счёт на \$200».

- 7. Kernel-уровень. BPF LSM или seccomp с проверкой по **SHA-256** — стережёт отпечаток бинаря, и на `execve`, и на `mmap`. Единственное, что реально бьёт самопатч на дне. Плюс аудит-лог во ВНЕШНИЙ приёмник (иначе агент подчистит).

Порядок действий

1. Включите PRIMARY-рычаги Hermes — это уже в продукте.
2. Поставьте spend limit в Console — снимает страх «счёта на \$200».
3. Оставляете агента автономно надолго — добавьте внешнюю изоляцию (слой 1) и strict sandbox (слой 2).
4. Kernel-слой (слой 7) — только если у вас свой Linux и вы понимаете, зачем.

Судья: контракт завершения (`/goal`) — чтобы «готово» значило «доказано»

Это тот самый судья из начала ролика. В **v0.18 «Judgment»** у `/goal` появились **контракты завершения**: вы описываете, как выглядит «сделано», и цикл цели засчитывает результат **по доказательству**, а не когда модели «показалось». Судья принимает только конкретную улику — результат команды, кусок файла, вывод тестов. [ОФИЦ v0.18 · docs Nous](#)

● ПРОСТО

вставьте этот промпт в Codex / Claude Code / Hermes

Поставь мне цель с контрактом завершения (`/goal`) для моего Hermes-агента. Я не разработчик.
ЗАДАЧА: <опиши своими словами, что нужно сделать>

- Пропиши контракт по пяти полям и покажи мне его ДО запуска:
1. OUTCOME — что должно стать правдой, когда задача закрыта (конкретно и проверяемо).
 2. VERIFICATION — ЧЕМ докажешь: какую команду выполнить / какой файл показать / какие тесты прогнать.
Судья засчитывает «готово» ТОЛЬКО по такой улике, а не по твоим словам.
 3. CONSTRAINTS — что трогать НЕЛЬЗЯ (файлы, конфиги, прод, деньги).
 4. BOUNDARIES — где тебе можно работать (папка / репозиторий / окружение), за пределы не выходить.
 5. STOP-CONDITION — при каком решении ты останавливаешься и спрашиваешь меня, а не импровизируешь.
- Дальше работай по контракту, а в конце покажи ДОКАЗАТЕЛЬСТВО, а не отчёт «сделано».

Руками — пять полей контракта

ПОЛЕ	ЧТО ЗАДАЁТ	ПРИМЕР
Outcome	Что должно стать правдой	«форма отправлена, письмо пришло на почту»
Verification	Чем доказать (улика)	вывод <code>curl</code> / ответ 200 / прогон теста
Constraints	Что менять нельзя	не трогать прод-конфиг, не ставить пакеты
Boundaries	Где работать	только папка <code>~/project/src</code>
Stop-condition	Когда спросить человека	если нужно потратить деньги или удалить данные

ПОЧЕМУ ЭТО И ЕСТЬ «СУДЬЯ»

Когда контракт задан, меняются оба промпта цикла: рабочий — целится в поверхность проверки и уважает ограничения, а **судейский** признаёт задачу закрытой только при выполненном критерии верификации **с конкретным доказательством**. В v0.18 Hermes ещё и **записывает свидетельства проверки** для кода и умеет закрыть задачу, реально прогнав ваши проверки.

ОФИЦ: docs Nous «Persistent Goals» + release v0.18.0 (2026.7.1)

ЧЕСТНАЯ ГРАНИЦА

Архитектурный принцип Hermes: агент вынесен Out of Scope — усмирение реализовано как **внешняя клетка**, а не самоограничение. Агента не просят вести себя хорошо — его помещают туда, где плохо себя вести технически нельзя. Слои 1–7 ниже PRIMARY-рычагов — это документированный стек Claude-Code-харнесса и практики изоляции ОС: тот же принцип «внешняя клетка», но включать их вы будете **на своей стороне ОС**, а не тумблером внутри Hermes. Слой 7 (BPF LSM / SHA-256) — рекомендация уровня ОС, не штатная кнопка. Сверьте синтаксис и применимость у себя.

ДОПОЛНИТЕЛЬНО • НАВЫКИ И ПРИЁМЫ

Что накрутить сверху: скиллы по ссылке и техники

Всё выше — база и родные фишки. Это — **то, что можно доставить сверху**: community-навыки, которые ставятся по ссылке (авторы указаны), и пара приёмов. Навыки в таблице — **сторонние**, не официальные кнопки. А вот часть техник ниже, наоборот, **официальные фишки Hermes** (Tool Search, слоёные промпты, `delegate_task`) — это помечено.

Честно про установку по ссылке. Сама механика установки навыка из хаба в коде **есть** (экран навыков, поиск по хабу, install). Но в текущем **dashboard-режиме (zero-fork)** маршрут установки/удаления навыка отдаёт **501 «only on legacy enhanced fork»** — то есть UI и код есть, но в этом режиме реально работают только **просмотр списка и вкл/выкл** уже установленных. Полная установка — на старом **legacy enhanced fork** либо руками через `SKILL.md`. Поэтому: пробуйте, но держите fallback.

● ПРОСТО

вставьте этот промпт в Codex / Claude Code / Hermes

Агент попробует найти и поставить сторонний навык — и честно скажет, если маршрут установки закрыт.

Помоги поставить сторонний навык (skill) в мой Hermes из хаба навыков.

НУЖЕН НАВЫК: <например «очеловечить русский текст» / «структурный веб-скрейпинг»>

- Открой экран навыков / поиск по хабу, найди подходящий сторонний навык.
- Попробуй установить. Если маршрут установки отдаёт ошибку 501 «legacy enhanced fork» или подобную — не выдумывай успех: скажи прямо, что автоустановка недоступна, и предложи ручной путь (положить SKILL.md вручную) либо альтернативу.
- Это сторонний навык, не официальный — предупреди меня и покажи, что он делает, прежде чем включать.

10 навыков, которые стоит поставить все — community, ставятся по ссылке · репозитории проверены 07.2026

НАВЫК	ЧТО ДЕЛАЕТ	ОТКУДА
youtube-skills	Транскрипты, поиск, данные каналов и плейлистов с любого YouTube-URL — без Google API и без блокировок на облачных IP. Чужое видео → транскрипт и темы за один запрос.	community · 341★
humanizer-ru (Rutext)	Очеловечивает русский текст агента — убирает «робота» из ответов. В ролике его называют «рутекст». Вариантов несколько; ссылка — на репозиторий из каталога <code>awesome-hermes-agent</code> , сверьте под свою версию.	community · peno: ilyautov
screenpipe	Пишет экран и звук 24/7 локально; агент помнит всё увиденное и сказанное, ищет по этому. «Что решили на созвоне?» — ответ, полностью на устройстве.	community · 20k★
hermes-web-search-plus	Заменяет встроенный поиск умным роутингом по Serper / Tavily / Eха: качественнее и разнообразнее источники, чистые URL — прямой апгрейд фактчекинга.	community · 343★
Agent Reach (web-scraper)	Отдаёт структурный текст вместо сырого HTML; экономнее по токенам (<code>~2,5×</code> — заявление автора на странице скилла). Вариантов несколько — ищите «Agent Reach» в каталоге или на GitHub, сверьте под свою версию.	community
mattpocock/skills	15 инженерных скиллов от автора Total TypeScript; <code>grill-me</code> допрашивает вас ПЕРЕД написанием кода — лекарство от «агент построил не то».	community · 178k★
gpt-image-2-agent-kit (дизайнер-агент)	Тулкит генерации картинок «из агента» (обложки и пр.). В ролике показан работающим (в демо — через Nano Banana, в самом репо — GPT Image 2).	community
hermes-studio	Панель управления Hermes: чат по нескольким бэкэндам, аналитика токенов и трат за 30 дней, крон-задачи, групповой мультиагент, встроенный терминал.	community · 9k★
SkillClaw	Авто-эволюция библиотеки навыков из реальных сессий: чистит дубли, улучшает. Пара к родному <code>/learn</code> .	community · 2k★
authsome	Локальный шлюз доступов: ключи в зашифрованном хранилище, агент работает с 45 сервисами (GitHub, Google, Stripe), не видя сырых секретов.	community · 71★

Техники

— **Слоёные промпты** ОФИЦ — подтверждённая архитектура prompt stack: `SOUL.md` / `AGENTS.md` / `MEMORY.md` / `skills`. Ядро тонкое, детали в отдельных файлах; агент подтягивает нужное, контекст не пухнет.

- **Tool Search (lazy-load инструментов)** **ОФИЦ** — официальная фича: инструменты MCP и плагинов превращаются в bridge-инструменты, их схема грузится **по требованию** и включается автоматически при росте контекста. Не подключать все схемы сразу.
- **Делегирование субагентам** **PRIMARY** — `delegate_task` реален (глубина ≤ 2), НО контекст **изолирован**: родитель видит только summary дочернего агента, не общую память. Это не «через общую вики».
- **Вики как источник истины** **community-приём** — отдельная схема ПАМЯТИ (не делегирования): индекс-вика находит нужное, но не заменяет источники истины.
- **Агент-интервью** **community-приём** — онлайн-строка в промпт агента: он сам расспрашивает вас и собирает профиль/ЦА, а не вы заполняете анкету.

ЧЕСТНАЯ ГРАНИЦА

Навыки в таблице — **community**, ставятся по ссылке, не официальные кнопки. Среди техник, наоборот, **официальные фичи Hermes**: слоёные промпты (prompt stack), Tool Search (lazy-load схем инструментов) и `delegate_task` — у него контекст изолирован, родитель видит только summary. «Вики как источник истины» и агент-интервью — community-приёмы (первое — про память, не про делегирование). Цифра «~2,5× токенов» у Agent Reach — заявление автора на странице скилла, не факт Hermes. Установка навыка по ссылке в коде есть, но в dashboard-режиме (zero-fork) маршрут установки/удаления отдаёт **501** — доступны только просмотр и вкл/выкл; полный install — на legacy enhanced fork или руками через `SKILL.md`. **Где брать сами скиллы**: community-навыки ставятся по ссылке — ищите по имени в открытом каталоге [awesome-hermes-agent](#) (или прямым поиском на GitHub) и ставьте по инструкции их репозитория. Это **community-каталог** (не официальный проект Nous): 100+ навыков плюс память, инструменты и мульти-агентные сборки, размеченные метками зрелости *production / beta / experimental* — сразу видно, что боевое, а что сырое. Его же авторы прямо предупреждают: попадание в список — подсказка «где искать», а не гарантия безопасности, поэтому любой навык проверяйте перед установкой. Честно: экосистема молодая (июль 2026) и разрозненная — у популярных имён вроде `humanizer-ru` есть несколько вариантов от разных авторов, берите по звёздам и репутации, а точный репозиторий из ролика сверяйте на его странице.

Доп материал (на изучение, не обязательно ставить). Отдельный community-проект **hermespace** — «карманный воркбенч + постоянная world-model» поверх Hermes: структурированная долгая память (убеждения с уровнем уверенности, события, причинные связи), которая растёт между сессиями и подмешивается в контекст, плюс дисциплина внимания «одна цель за раз» и плагин для Hermes. Чистый Python, ноль зависимостей, MIT. Честно: проект свежий (июль 2026), соло-автор, в реальной эксплуатации ещё не обкатан — берите как «куда движется идея постоянной памяти для агента», а не как проверенную готовую инструкцию. Репозиторий: github.com/PabloTheThinker/hermespace.

СВЕЖЕЕ · 0.19 «QUICKSILVER»

Что прилетело прямо сейчас — обновление 0.19

Пока снимали, вышло обновление **0.19 «Quicksilver»** (20 июля 2026, по официальным release notes). Больших новых кнопок немного — это апдейт про **скорость и надёжность**. Что это даёт лично вам:

- **Первый ответ — почти на 80% быстрее.** Холодный старт упал с ~4,3 с до ~0,9 с — меньше ждёте перед первой репликой.
- **Мысли модели видно вживую.** Размышления стримятся по мере генерации — не смотрите в пустой спиннер по полминуты.
- **Умные одобрения — по умолчанию.** Опасную команду сначала оценивает отдельная модель-ревьюер, а не только чёрный список: меньше лишних вопросов по мелочи, жёстче реакция на реально опасное.

- **Пароли — в менеджерах.** Ключи можно тянуть из Bitwarden или 1Password (`op://` -ссылки), а не держать россыпью в файлах.
- **Надёжная фоновая делегация.** `delegate_task` отдаёт живой лог-файл (`tail -f`) — сразу видно, как суб-агенты делают работу, а не гадаете, живы ли они.

ИСТОЧНИК

Официальные release notes **v0.19.0 «The Quicksilver Release»** (GitHub NousResearch/hermes-agent, сверено 20.07). Ключевые фишки этого гайда — судья, память, навыки — пришли раньше, в 0.18 «Judgment». Хайп-цифры из обзоров («web search 60× быстрее», «маркетплейс навыков») в официальных release notes 0.19 не значатся — не берём.

05 · ЧЕСТНЫЕ ОГОВОРКИ

Четыре вещи, которые важно держать в голове

Материал специально без «успешного успеха». Четыре оговорки, которые отделяют то, что работает уже сейчас, от того, что ещё в пути или подано прессой.

ОГОВОРКА 1 · ОБЛАКО

Кросс-девайс синхронизация — «Coming Soon»

Официальная кросс-девайс синхронизация, командная общая память и one-click deploy у самих Nous помечены в README **Status: Coming Soon / Pending infra**. Кнопки, которой пока нет, не ищите. Что работает уже сейчас — один инстанс + общее хранилище + Tailscale (Тема 3).

ОГОВОРКА 2 · ТАРИФ CLOUD

\$0,06/день в простое · \$0,32–1,12/день активный

Официальное Hermes Cloud (PREVIEW, v0.18.2, регион lhr) — оплата по дням, scale-to-zero: Small \$0,32 / Medium \$0,59 / Large \$1,12 в день активной работы, и **\$0,06/день в простое для всех размеров** (это минимальная плата за хранение, не ноль). Для запуска нужен минимум \$10 кредитов. Ставки **не включают** inference и инструменты — списываются отдельно. Сверяйте на день установки.

ОГОВОРКА 3 · GLM 5.2

Не самая умная — но ≈6× дешевле

GLM 5.2 проигрывает флагманам по «сырому уму» (Opus 4.8 и GPT-5.5 выше на Terminal-Bench 2.1). Её ценность — **баланс «ум на деньги»**: \$1,40/\$4,40 против \$5/\$30 у флагмана OpenAI, то есть **≈6–7× дешевле по выходным токенам**. «1/6 стоимости» — это фрейминг прессы, а не заявление Z.ai. Все бенчмарки — «как сообщается вендором».

Claude — только API-ключ; ChatGPT — можно подпиской

Разводим, потому что путают постоянно. Подписочная OAuth-квота **Claude** (Claude Pro/Max) с апреля 2026 работает только в нативных приложениях Anthropic — Hermes как сторонней обёртке нужен отдельный **Anthropic API-ключ**. А вот **ChatGPT Plus/Pro** подключается в **Hermes ПО ПОДПИСКЕ** — через штатный провайдер **OpenAI Codex** (вход по OAuth-коду на auth.openai.com/codex/device, без API-ключа; модель GPT-5.6). Единственный минус — недельный кап Codex-подписки: круглосуточный тяжёлый агент может его выбрать. Поэтому базовый рабочий конь в видео — GLM/DeepSeek (без сюрпризов), а ChatGPT-подписка — рабочий бонус. Статус OAuth подвижен — перепроверьте на день установки.

Забрали → поставили сегодня. Два пути на каждую тему: не хотите YAML — вставляете жёлтый промпт и агент делает всё сам; хотите понимать каждую строчку — берёте конфиг руками. Начните с той темы, которая болит прямо сейчас, — остальные три подождут.

— Эдвард Гришин, Futura AI

Что дальше. Все конфиги из видео — бесплатно, файлом, в Telegram-боте под роликом. Вместе с этим гайдом там лежит файл `промнты-Codex-Claude.md` : те же четыре промпта плюс **мастер-промт «настрой всё сразу»** — вставили один раз, и агент проходит модель + память + клетку по шагам с подтверждением. Части 1 и 2 серии — по ссылкам в описании. Разбор целиком, с упаковкой ИИ-внедрения в продукт — в Академии FuturaAI.